

Lab 3: Decomposing your series

MATH840 | Time Series | Week 3 — graded assignment 2 of 7

AUTHOR

Ihor Miroshnychenko

AFFILIATION

Kyiv School of Economics

 Open the notebook

 Open in Colab

Open In Colab

Click **Copy to Drive** before you start working, otherwise nothing you write is saved.

 Want to see it done first?

[Worked example](#) — the whole assignment, start to finish, on Queensland clothing retailing. That series is drawn from `aus_retail`, which is **not** in the pool your own series comes from, so it demonstrates the method without handing you any of your answers.

The task

You have a series, and in Lab 2 you found out which of the four simple methods wins on it. This week you find out **why**.

Concretely: stabilise its variance if it needs it, decompose it two ways and compare the two, quantify how strong its trend and seasonality are, and explain last week's winner in terms of those components.

 **This one counts**

8 points, produced and submitted **within the session**. Deadline is the end of the practice slot; there is no homework phase. Work is marked afterwards and returned before the next practice session.

What you are marked on

Section	Criteria	Points
1	Data preparation	1
2	EDA and visualisation	4
3	Implementation	2
4	Code quality and reproducibility	1

Section	Criteria	Points
Total		8

The notebook has one section per row. **Keep those headings.** They are how the work is marked, and a report that reorders or drops them is slower to read and scores worse for it.

Your series

The same one you have used since Week 1 — the same student identifier produces the same dataset. If you have lost your notebook, the assignment function is in the template again.

Warning

Do not switch datasets. This is the same series you used last week: last week you found out which simple method wins on it, this week you explain why.

What is not marked again

Loading your series and repairing its calendar were marked in Lab 2, so they are not marked twice. The template prepares the series for you exactly the way Lab 2 asked — duplicates averaged, a regular calendar, a missing tail trimmed, interior gaps interpolated. If you made different choices in Lab 2, replace that cell with your own code and carry on.

Your Lab 2 graphics are not repeated either. Refer back to them whenever they help an explanation — they are yours.

Suggested pacing (80 minutes)

Minutes	What
0–10	Setup: open the template, load your prepared series
10–25	Section 1: does the variance need stabilising? Decide and justify
25–55	Section 2: classical decomposition, STL, the two compared, the seasonally adjusted series
55–70	Section 3: F_T , F_S , and the explanation of last week's winner
70–80	Export and submit

If you are behind at minute 70, **submit what you have**. A valid partial submission scores; a perfect missing one does not.

Section 1 — Data preparation (1 point)

Does the variance need stabilising? Does the size of the seasonal swing grow with the level of the series? If it does, the additive decompositions in Section 2 are the wrong model for it.

- Estimate a Box-Cox λ (`boxcox_lambda` from `coreforecast`, as in the lecture).
- Plot the transformed series next to the original.
- **Decide** whether to work transformed or untransformed, and justify the decision in a sentence. A defensible “no transformation needed” is worth full marks; an unexplained transformation is not.

 `boxcox` / `boxcox_lambda` · `scipy.stats.boxcox` (an alternative that returns the transform and λ together) · [FPP 3.1 transformations](#) · [FPP the Pythonic Way, ch. 3](#)

If your series contains zeros

`boxcox_lambda(..., method="loglik")` refuses non-positive values outright. `method="guerrero"` accepts them but may return $\lambda = 0$, which *is* the log transform — undefined at zero. A multiplicative decomposition refuses them for the same reason.

That is information about your series, not an obstacle. Find out where the zeros come from before deciding what to do about them, and say what you concluded. This is exactly the kind of judgement the point is for.

Section 2 — EDA and visualisation (4 points)

Two decompositions, their comparison and an adjusted series — one point each, all on the version of the series you settled on in Section 1. Every plot needs a reading: a plot with no reading attached earns nothing.

Item	What earns the point
2.1 Classical decomposition	<code>seasonal_decompose</code> , and a reading of its components
2.2 STL	STL with a stated choice of <code>seasonal</code> and <code>robust</code> and the reason for it, and a reading of its components
2.3 Classical versus STL	where the two disagree, and which you trust more on this series — with the reason visible in the plots

Item	What earns the point
2.4 Seasonally adjusted series	the series with the STL seasonal component removed: what it is useful for, and what it hides

⚠️ Where this goes wrong

- `seasonal_decompose` returns `NaN` for the first and last `period / 2` observations — six months at each end of a monthly series. That is the centred moving average, not a bug, and it is one of the differences 2.3 asks you to comment on.
- `model="multiplicative"` refuses zeros and negative values outright.
- `STL(...)` needs `seasonal` to be an **odd** integer ≥ 3 , and it must be an `int`, not a float. `robust=True` costs a little accuracy and buys resistance to outliers.
- Both need `period` — the number of observations in one cycle, which is `mine["season"]`, not the number of years.

📖 [seasonal_decompose](#) · [STL](#) · [FPP 3.4 classical decomposition](#) · [FPP 3.6 STL](#)

📖 What “reading the components” means

Not “the trend goes up”. Something a colleague could act on: *the seasonal component is stable until 2008 and shifts afterwards, the remainder is flat except for three spikes which line up with the crisis, the trend flattens in the last two years.*

134 Section 3 — Implementation (2 points)

3.1 Strength of components (1 point). Compute, from the STL fit:

$$F_T = \max \left(0, 1 - \frac{\text{Var}(R_t)}{\text{Var}(T_t + R_t)} \right) \quad F_S = \max \left(0, 1 - \frac{\text{Var}(R_t)}{\text{Var}(S_t + R_t)} \right)$$

Report both, rounded to two decimals.

📖 [FPP 4.3 — where \$F_T\$ and \$F_S\$ come from](#)

Both come out of a **fitted** STL object: `fit.trend`, `fit.seasonal`, `fit.resid`. If you get a value outside $[0, 1]$, you have almost certainly passed the STL object instead of the result of `.fit()`.

3.2 Explaining last week’s winner (1 point). In Lab 2 you found which simple method won on your series. Now explain **why**, with the decomposition.

- Set `LAB2_WINNER` to the method that won in your Lab 2 — it is in your `SURNAME_lab02_summary.csv`.
- In three or four sentences, explain that result using F_T , F_S and what the components show: which component carries the variance, and what the winning method copies forward that the other three do not.
- If your Lab 2 prediction was wrong, say what the decomposition shows that the plots did not.

⚠ Important

Marked on the **reasoning**. “snaive won because snaive usually wins” scores nothing, even though it is usually true.

ℹ Missed Lab 2?

The template has an optional cell that recomputes the four methods on the same holdout Lab 2 used. Run it after the setup cell, and use its winner.

🔪 Section 4 — Code quality and reproducibility (1 point)

- The notebook runs top to bottom on a fresh kernel without errors.
- No dead cells, no leftover experiments, no `df2_final_v3`.
- Every number in your text comes from a cell, not from your memory of an earlier run.

📁 Submission

Upload three files to the Week 3 activity on Moodle **before the end of the session**:

- `SURNAME_lab03.pdf` — the rendered report;
- `SURNAME_lab03.ipynb` — the source;
- `SURNAME_lab03_summary.csv` — written by the last cell of the template: your identifier, F_T , F_S and last week's winner.

📄 For next week

Next week's lab is built on today's lecture, the forecaster's toolbox. Read Chapter 5 of [Forecasting: Principles and Practice, the Pythonic Way](#) before then — fitted values and residuals, residual diagnostics, prediction intervals and accuracy evaluation.